

Task 42: Vertical Order Traversal of a Binary Tree

Problem Description

Given the root of a binary tree, calculate the vertical order traversal of the binary tree. For each node at position (row, col), its left and right children will be at positions (row + 1, col - 1) and (row + 1, col + 1) respectively. The root of the tree is at (0, 0). The vertical order traversal of a binary tree is a list of top-to-bottom orderings for each column index starting from the leftmost column and ending on the rightmost column. There may be multiple nodes in the same row and same column. In such a case, sort these nodes by their values.

Java Solution

```

class Solution {
    class Tuple {
        TreeNode node;
        int row;
        int col;
        public Tuple(TreeNode node, int row, int col) {
            this.node = node;
            this.row = row;
            this.col = col;
        }
    }

    public List<List<Integer>> verticalTraversal(TreeNode root) {
        TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>> map =
new TreeMap<>();
        Queue<Tuple> q = new LinkedList<>();
        q.offer(new Tuple(root, 0, 0));

        while (!q.isEmpty()) {
            Tuple tuple = q.poll();
            TreeNode node = tuple.node;
            int x = tuple.row;
            int y = tuple.col;

            if (!map.containsKey(x)) {
                map.put(x, new TreeMap<>());
            }
            if (!map.get(x).containsKey(y)) {
                map.get(x).put(y, new PriorityQueue<>());
            }
            map.get(x).get(y).offer(node.val);

            if (node.left != null) {
                q.offer(new Tuple(node.left, x - 1, y + 1));
            }
            if (node.right != null) {
                q.offer(new Tuple(node.right, x + 1, y + 1));
            }
        }

        List<List<Integer>> list = new ArrayList<>();
        for (TreeMap<Integer, PriorityQueue<Integer>> ys : map.values()) {
            list.add(new ArrayList<>());
            for (PriorityQueue<Integer> nodes : ys.values()) {

```

```
        while (!nodes.isEmpty()) {
            list.get(list.size() - 1).add(nodes.poll());
        }
    }
    return list;
}
}
```